

Design a Cache

Composition over Inheritance · Replacing Conditionals with Polymorphism · Factory · TTL & Decorator Limits

1. The Problem — Design an In-Memory Cache

Requirements:

- Key-value store, fast ($O(1)$) `get` / `put` / `remove`.
- Bounded **capacity** — when full, something must be **evicted**.
- Eviction policy varies: FIFO, LRU, LFU, MRU, Random, Second-Chance... and the team will keep inventing more.
- Later: TTL (entries expire), metrics, logging.

```
public interface Cache<K, V> {
    V get(K key);           // (we'll revisit this signature - what's our null rule?)
    void put(K key, V value);
    V remove(K key);
    boolean containsKey(K key);
    int size();
}
```

The volatile part is obvious from the requirements: **the eviction policy**. Everything else — store, look up, count — is stable. You know the rule by now: *find what keeps changing, and isolate it*.

2. Design 1 — The Naive Version (one class per policy)

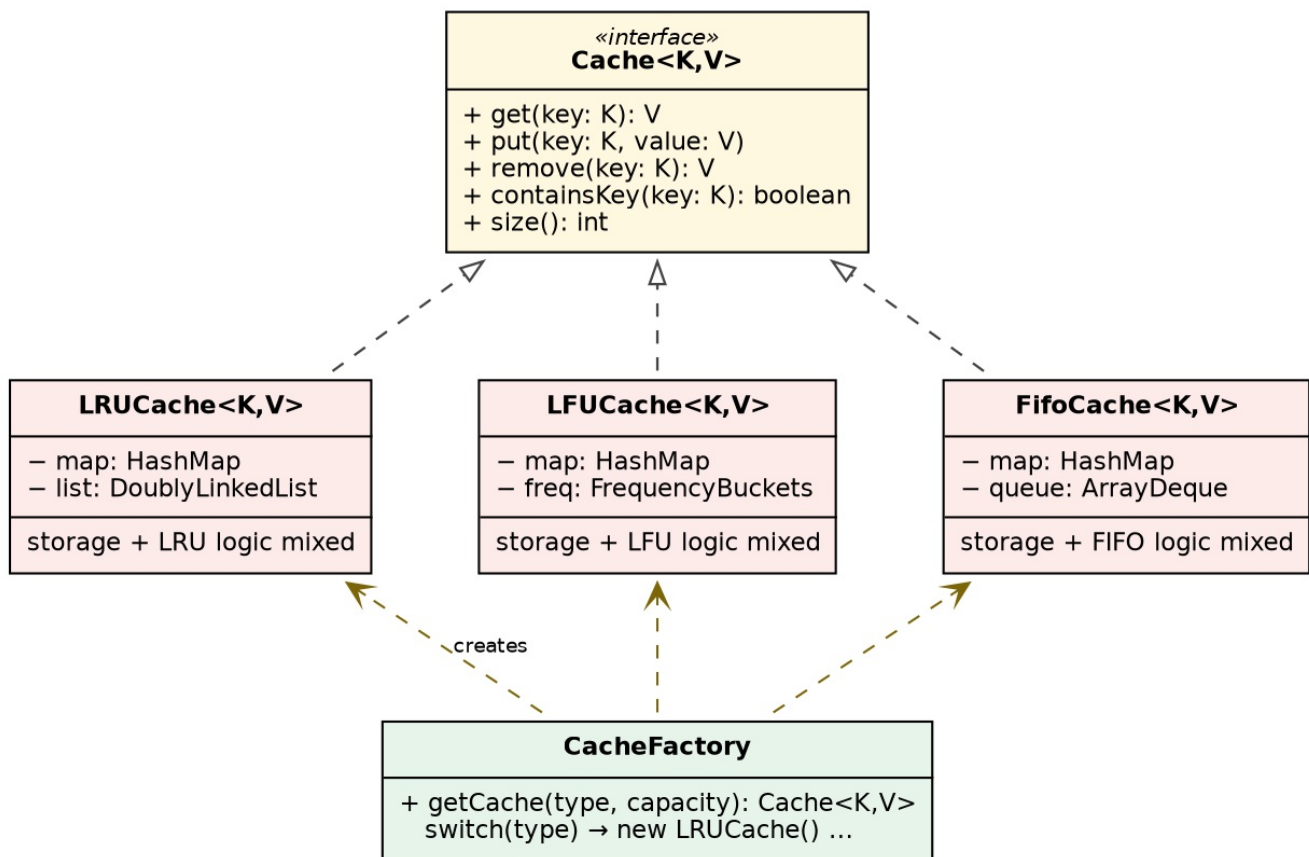
The first instinct:

```
class LRUCache<K, V> implements Cache<K, V> { /* map + linked list + LRU logic */ }
class LFUCache<K, V> implements Cache<K, V> { /* map + frequency buckets */ }
class FIFOCache<K, V> implements Cache<K, V> { /* map + queue */ }

// and creation:
class CacheFactory {
    static <K, V> Cache<K, V> getCache(String type, int capacity) {
        return switch (type) {
            case "lru" -> new LRUCache<>(capacity);
            case "lfu" -> new LFUCache<>(capacity);
            case "fifo" -> new FIFOCache<>(capacity);
            default -> throw new IllegalArgumentException("unknown: " + type);
        };
    }
}
```

This *works*, and for an interview warm-up it's fine. But watch it strain:

1. **Each class re-implements storage.** The `HashMap` handling, the capacity check, the `size()` / `containsKey()` boilerplate — copy-pasted into every cache. The policies differ in ~30 lines; the classes duplicate ~100.
2. **Two concerns in one class.** `LRUCache` mixes *how entries are stored* with *which entry dies next*. Two reasons to change, one class — SRP says split.
3. **The feature matrix explodes.** Now add TTL: `LRUCacheWithTTL`, `LFUCacheWithTTL` ... Add metrics: `LRUCacheWithTTLAndMetrics` ... N policies $\times$ M features = $N \times M$ classes. We've met this exponent before (constructors: 2^n ; I/O streams: 2^n). Inheritance multiplies; composition adds.



Design 1 — one class per policy: each mixes storage with eviction logic, and the factory creates concretes.

But first — the LRU mechanics (interview must-know)

Whatever the design, LRU itself needs $O(1)$ `get` and $O(1)$ reorder. The classic structure: **HashMap + doubly linked list**.

```

most-recent → [4|val] ⇌ [3|val] ⇌ [2|val] ← least-recent (evict from here)
map: { 2 → node2, 3 → node3, 4 → node4 }    (key → NODE ADDRESS, not value)
  
```

- `get(k)` : map gives the node in $O(1)$ → unlink it → move to head. The map stores *node references* precisely so we never scan the list.
- `put(k,v)` : at capacity → unlink the tail (that's the LRU victim), remove from map → insert new node at head.
- Every operation $O(1)$. (Java's `LinkedHashMap(capacity, loadFactor, accessOrder=true)` is literally this structure, with a `removeEldestEntry` hook — the JDK ships an LRU cache in disguise.)

3. Design 2 — Composition: Storage ⊕ EvictionPolicy

Split the two concerns into two interfaces, and let the cache **have** them:

```

public interface CacheStorage<K, V> {           // concern 1: where entries live
    V get(K key);
    void put(K key, V value);
    V remove(K key);
    boolean containsKey(K key);
    int size();
}

public interface EvictionPolicy<K> {           // concern 2: who dies next
    // hooks — the cache NOTIFIES the policy
    void onGet(K key);
    void onPut(K key);
    void onRemove(K key);
    K selectVictim();
}
  
```

Note the shape of `EvictionPolicy`: it never touches values or storage. It only *observes* access events (`onGet`, `onPut`,

onRemove) and answers one question (selectVictim). LRU implements it with the linked list; LFU with frequency counts; Random with a set and a dice roll — the cache neither knows nor cares.

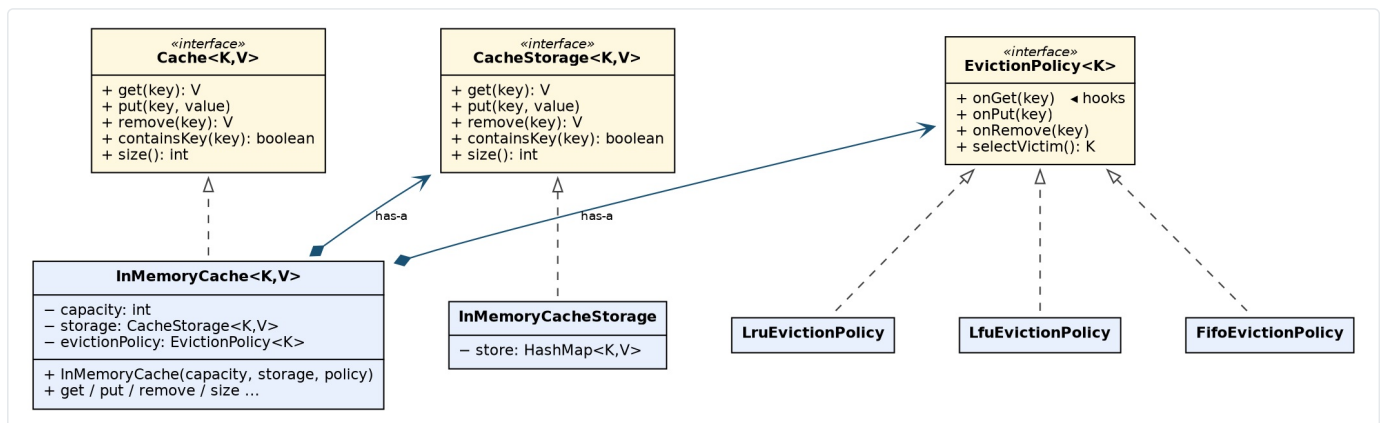
```
public class InMemoryCache<K, V> implements Cache<K, V> {
    private final int capacity;
    private final CacheStorage<K, V> storage; // HAS-A, not IS-A
    private final EvictionPolicy<K> evictionPolicy; // HAS-A, not IS-A

    public InMemoryCache(int capacity, CacheStorage<K, V> storage, EvictionPolicy<K> policy) {
        this.capacity = capacity; // dependency injection — as always
        this.storage = storage;
        this.evictionPolicy = policy;
    }

    @Override
    public V get(K key) {
        V value = storage.get(key);
        if (value != null) {
            evictionPolicy.onGet(key); // hook: "this key was touched"
        }
        return value;
    }

    @Override
    public void put(K key, V value) {
        boolean exists = storage.containsKey(key);
        if (!exists && storage.size() == capacity) {
            K victim = evictionPolicy.selectVictim(); // policy decides WHO
            storage.remove(victim); // cache decides WHAT HAPPENS
            evictionPolicy.onRemove(victim);
        }
        storage.put(key, value);
        evictionPolicy.onPut(key);
    }

    // remove/containsKey/size delegate to storage (+ onRemove hook) ...
}
```



Design 2 — the cache HAS-A storage and HAS-A eviction policy; both vary independently behind interfaces.

What this buys, in the vocabulary we've built:

- **Strategy:** eviction is a pluggable decision — swap LRU→LFU by changing one constructor argument, put() untouched.
- **Template-method flavor:** get / put define the fixed skeleton; the variable steps are delegated through hooks. (Classically template method uses subclass overrides; we use an injected object — composition again.)
- **SRP:** storage changes for storage reasons, policy for policy reasons, the cache class only for orchestration changes.
- **Testability:** test LruEvictionPolicy alone by firing hook events at it — no map, no cache, no values.
- **The N×M explosion dies:** N policies + M storages = N + M classes, combined at the composition root.

(Interface nit, carried from Notes 03: V get() returning null to mean "miss" violates our no-null rule. A stricter interface is Optional<V> get(K key) — decide it consciously; real cache APIs like Caffeine return null here for performance and document it loudly. Knowing you're breaking a rule, and why, is different from not knowing the rule.)

4. Composition over Inheritance — the full argument

The principle: to reuse or extend behavior, prefer *containing* an object (HAS-A) and delegating, over *extending* a class (IS-A).

4.1 Why inheritance is the strongest — and most dangerous — coupling

A subclass depends on its parent's fields, methods, **and undocumented internal behavior**. The canonical demonstration (from *Effective Java*, Item 18 — this is real JDK behavior, run it):

```
public class InstrumentedHashSet<E> extends HashSet<E> {
    private int addCount = 0;

    @Override public boolean add(E e) {
        addCount++;
        return super.add(e);
    }

    @Override public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c);
    }

    public int getAddCount() { return addCount; }
}

var s = new InstrumentedHashSet<String>();
s.addAll(List.of("a", "b", "c"));
s.getAddCount();           // 6. SIX. Not 3.
```

Why? Because `HashSet.addAll` internally calls `this.add(...)` — which is now *your* override — so every element counts twice. Your class's correctness silently depends on an **implementation detail of the parent** that no contract promised and any JDK update may change. This is the **fragile base class problem**, and it's why "extends" is a bigger commitment than it looks.

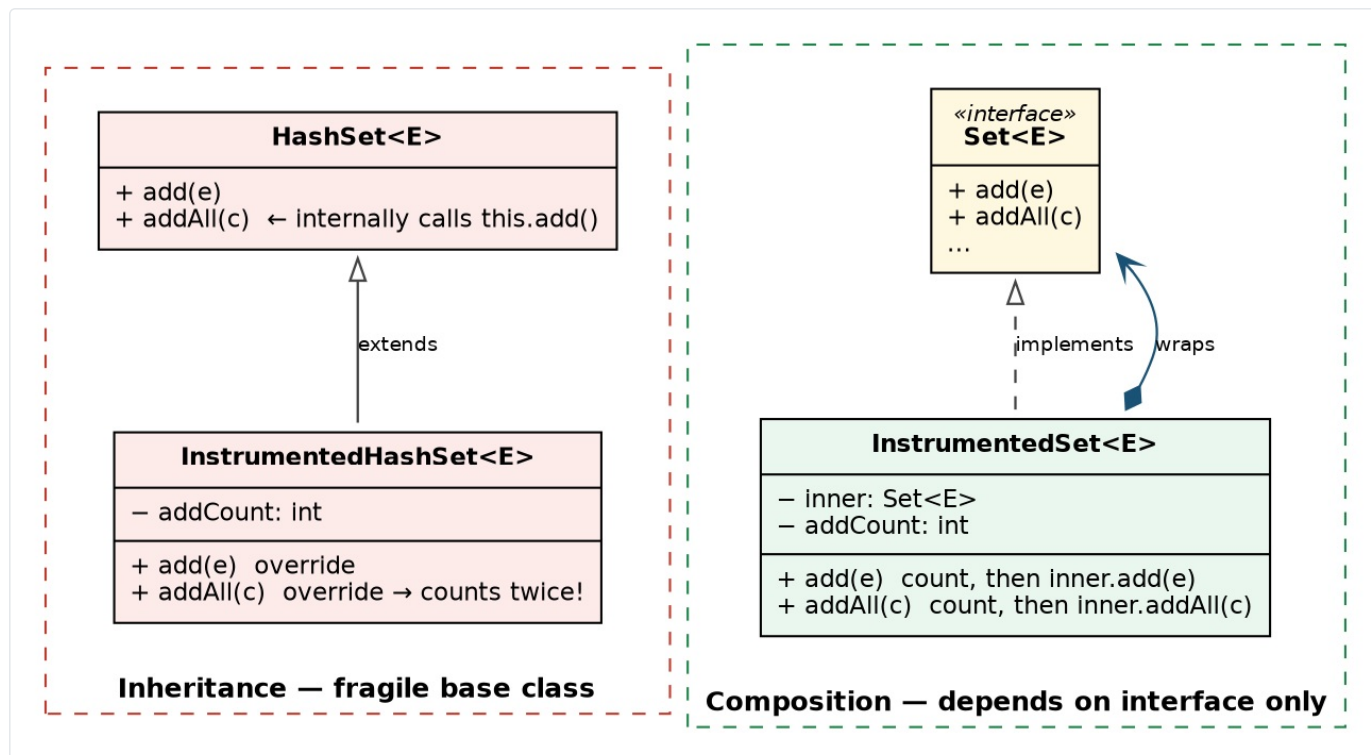
The composition fix — wrap, don't extend:

```
public class InstrumentedSet<E> implements Set<E> {
    private final Set<E> inner;           // HAS-A
    private int addCount = 0;

    public InstrumentedSet(Set<E> inner) { this.inner = inner; }

    public boolean add(E e) { addCount++; return inner.add(e); }
    public boolean addAll(Collection<? extends E> c) { addCount += c.size(); return inner.addAll(c); }
    // everything else: delegate to inner ...
}
```

Now the count is right *no matter how* the inner set implements `addAll` — you depend on its **interface**, not its internals. (Also notice: this wrapper works for `HashSet`, `TreeSet`, `LinkedHashSet`... the inheritance version instrumented exactly one.) And yes — this wrapper is a **decorator**; composition over inheritance is the soil decorator grows in.



Same goal, two structures: the subclass inherits the parent's internals; the wrapper depends only on the interface.

4.2 The JDK's own regrets (real, shipping, permanent)

The Java standard library contains famous inheritance mistakes, kept forever for backward compatibility:

```
// java.util.Stack extends Vector – so this compiles:
Stack<Integer> stack = new Stack<>();
stack.push(1); stack.push(2); stack.push(3);
stack.add(1, 99);           // insert into the MIDDLE of a "stack" – LIFO destroyed
stack.remove(0);           // remove the BOTTOM
```

```
// java.util.Properties extends Hashtable<Object, Object> – so this compiles:
Properties props = new Properties();
props.put("timeout", 42);           // an Integer into a String-only store
props.getProperty("timeout");       // → null. The value is there; the API can't see it.
```

A `Stack` is not usable-everywhere-a-`Vector` is expected to be *mutated like one* — inheritance exposed 40 parent methods that break the child's own invariant. The modern JDK docs themselves say to use `ArrayDeque` (which *composes* an array) instead. When the JDK's own authors list these as design regrets, believe them.

4.3 What composition wins — the complete scorecard

	Inheritance (IS-A)	Composition (HAS-A)
Coupling	to parent's internals (fragile base)	to a public interface only
Chosen when	compile time , fixed forever	runtime — inject/swap/reconfigure
Combining behaviors	one parent only; N×M subclass explosion	mix freely: N + M pieces
Testing	must drag the whole parent along	inject a fake — test in isolation
Encapsulation	parent's protected guts leak in	inner object stays sealed
LSP risk	every subclass is a substitutability claim	no claim made

4.4 Real-world composition, everywhere you look

- `Collections.unmodifiableList(list)`, `synchronizedList(list)` — the JDK adds immutability and thread-safety by *wrapping*, not by `UnmodifiableArrayList extends ArrayList` (which would need unmodifiable variants of every list class — the N×M explosion again).
- Caffeine** (the standard high-performance Java cache — this is our exact design, industrial-grade):

```
Cache<String, User> cache = Caffeine.newBuilder()
    .maximumSize(10_000)           // eviction (W-TinyLFU inside)
    .expireAfterWrite(Duration.ofMinutes(5)) // expiration policy
    .recordStats()                 // metrics
    .build();
```

No `BoundedExpiringRecordingCache` class exists — size policy, expiration policy, and stats are **composed capabilities** configured at build time. Three of our lecture topics (Strategy, composition, Builder) in four lines of a real library. - **Our own codebase:** `InMemoryCache` has-a storage and has-a policy; `QuestionService` has-a repository; `TextProcessor` has-a composed function. We have not written `extends` once in five classes — and haven't missed it.

4.5 When inheritance IS the right tool

Composition-over-inheritance is a default, not a ban. Inheritance earns its place when **all** of these hold: the relationship is genuinely and permanently IS-A; the subclass upholds every behavioral contract of the parent (LSP — remember Rectangle/Square); and you own or the framework mandates the hierarchy (`extends HttpServlet`, `extends RecyclerView.Adapter`, our `QuestionRepository` `extends CrudRepository` — an *interface* hierarchy, which shares only contract, never fragile implementation). One honest sentence to carry: **inherit for polymorphic substitution, compose for reuse**. Reusing code via `extends` is the misuse; being substitutable via an interface is the point.

5. Replace Conditionals with Polymorphism — and the Honest Accounting

5.1 The class example, sharpened

```
enum CustomerType { BASIC, SILVER, GOLD, PLATINUM }

// the conditional version
public double calculateSubscriptionFee(Customer customer) {
    if (customer.type() == BASIC)    return 0.0;
    if (customer.type() == SILVER)   return customer.price();
    if (customer.type() == GOLD)     return customer.price() * 1.2;
    if (customer.type() == PLATINUM) return customer.price() * 2.0;
    throw new IllegalStateException();
}
```

One if-chain is harmless. The disease starts when the *same* chain replicates across the codebase — because customer type never influences just one thing:

```
calculateSubscriptionFee(...) // if BASIC... if SILVER... if GOLD...
supportResponseTime(...)    // if BASIC... if SILVER... if GOLD...
maxDevicesAllowed(...)      // if BASIC... if SILVER... if GOLD...
renderBadge(...)            // if BASIC... if SILVER... if GOLD...
```

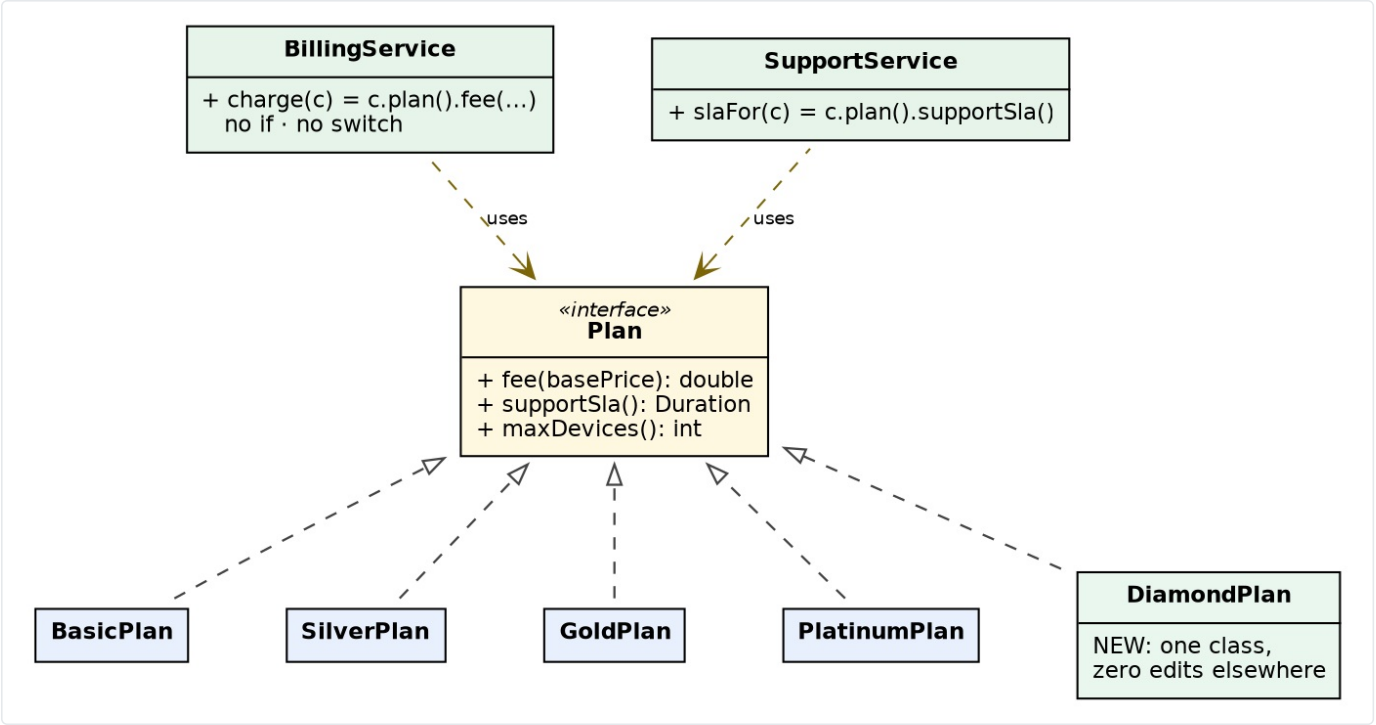
Now add DIAMOND. You must find and correctly update **every** chain — and some were "optimized" (`if type != BASIC` shortcuts), some live in another team's module. Miss one and DIAMOND customers get BASIC support silently. This is exactly the shapes-switch disease from Notes 01, wearing a subscription tier.

The polymorphic version — one type, one home for ALL its behavior:

```
public interface Plan {
    double fee(double basePrice);
    Duration supportSla();
    int maxDevices();
}

public class BasicPlan implements Plan {
    public double fee(double p) { return 0.0; }
    public Duration supportSla() { return Duration.ofHours(48); }
    public int maxDevices() { return 1; }
}
public class GoldPlan implements Plan { ... }
public class PlatinumPlan implements Plan { ... }

// domain code – conditional-free, forever:
public class BillingService {
    public double charge(Customer c) {
        return c.plan().fee(c.basePrice()); // no if. no switch. dispatch does it.
    }
}
```



All tier behavior lives with the tier; services dispatch through the interface — conditional-free.

Adding DIAMOND = one new class implementing Plan, plus one registration line. BillingService, SupportService, DeviceService — none of them change, none even recompile.

5.2 A real one you use daily: JDBC

Your code:

```
Connection conn = DriverManager.getConnection("jdbc:postgresql://prod:5432/app");
PreparedStatement st = conn.prepareStatement("SELECT ...");
```

Question: **where is the if (database == POSTGRES) ?** Nowhere in your code — ever. Connection, PreparedStatement, ResultSet are interfaces; the object behind conn is org.postgresql.jdbc.PgConnection, a class you never import and never name. Change the URL to jdbc:mysql://... and a completely different implementation flows through the same code. Millions of applications run database-vendor-conditional-free because the conditional was paid once, inside a factory (DriverManager), and polymorphism carries the decision everywhere else. The same trick powers Calendar.getInstance(locale), Charset.forName("UTF-8"), SLF4J picking its logging backend, and Spring choosing implementations by config.

5.3 THE honest question: did the conditional disappear, or just move?

It moved. And that's the entire point. Be precise about what changed, because "polymorphism removes conditionals" is sloppy — here is the true accounting:

	Before (if-chains)	After (polymorphism)
How many places choose?	N sites — every method that varies by type	1 site — where the object is created (factory / composition root / map)
How often does the choice run?	on every call, at every site	once, at construction; then dispatch is automatic
What holds the decision?	control flow, duplicated	data — the object's type, chosen once and carried
Adding a type touches	every chain (find them all — good luck)	one new class + one registration line
Branch logic testable alone?	no — entangled in host methods	yes — each class in isolation

So the conditional still exists — look inside CacheFactory and there it is, a switch. But it exists once, at the edge, in the least interesting file in the system, instead of smeared through the domain. And even that last switch can dissolve into pure data:

```
// the switch, turned into a lookup table — creation is now data, not control flow
private static final Map<String, Supplier<EvictionPolicy<Object>>> POLICIES = Map.of(
    "lru",      LruEvictionPolicy::new,
    "lfu",      LfuEvictionPolicy::new,
    "fifo",     FifoEvictionPolicy::new,
    "random",   RandomEvictionPolicy::new);

// registering a new policy = one line here. No branch anywhere runs at request time.
```

One layer of honesty deeper: polymorphic dispatch **is itself a conditional** — the vtable from Notes 01 is a machine-made jump table. We never eliminated *choice*; we moved it from hand-written, duplicated control flow into a table the compiler maintains and can never miss a case of. "Replace conditionals with polymorphism" really means: **choose once, as data, at the edge — instead of repeatedly, as code, in the middle.**

5.4 ...and when the switch should stay

Don't let the refactoring become a reflex. From our own codebase, `VoteService.applyDelta`:

```
switch (postType) {
    case QUESTION -> questions.incrementVoteCount(postId, delta);
    case ANSWER   -> answers.incrementVoteCount(postId, delta);
}
```

The rule of thumb, as a 2×2:

- **Few cases × one site** → keep the exhaustive `switch` over an enum. The compiler polices it; a class hierarchy would be ceremony. (*applyDelta*, our *SortBy* comparator.)
- **Cases multiply OR sites multiply** → polymorphism. The moment you write the *second* switch over the same enum, or the *fifth* case, the balance tips. (*Plans*, *shapes*, *eviction policies*.)

Modern Java sweetens the switch side of this trade: an exhaustive `switch` over an enum or sealed interface **fails to compile** when a new case appears — the compiler hunts down every site for you. That's a real answer to "we'll miss one," and it's why the honest modern rule is *sites*, more than *cases*: duplicated dispatch is the disease; a single well-policed switch is fine.

6. The Factory Pattern — why, when, and the questions you're about to ask

6.1 The reasoning from first principles

`new` is the most innocent-looking coupling there is. `new LruEvictionPolicy<>()` in a class means that class **names a concrete type** — it can never be handed an LFU, never be tested with a fake, and recompiles when the concrete changes. Everything we've built says: push concrete choices to the edge. A **factory is where the edge puts them** — one component whose only job is *deciding which concrete thing to build and assembling it*.

What a factory encapsulates, concretely:

1. **The mapping from intent to implementation.** `"lru"` (a config string, a user choice, an enum) → `LruEvictionPolicy`. Clients express intent; the factory knows the classes.
2. **The assembly recipe.** Our real cache isn't one `new` — it's capacity + storage + policy wired together. The factory owns the recipe; callers get a finished object:

```
public final class Caches {
    public static <K, V> Cache<K, V> of(String policy, int capacity) {
        EvictionPolicy<K> p = switch (policy) {
            case "lru" -> new LruEvictionPolicy<>();
            case "lfu" -> new LfuEvictionPolicy<>();
            case "fifo" -> new FifoEvictionPolicy<>();
            default -> throw new IllegalArgumentException("unknown policy: " + policy);
        };
        return new InMemoryCache<>(capacity, new InMemoryCacheStorage<>(), p);
    }
}

Cache<Long, Question> cache = Caches.of(config.get("cache.policy"), 10_000);
```

3. **The return type is the interface.** Callers hold `Cache<K, V>`, never `InMemoryCache`. The factory is DIP applied to the *moment of creation* — the one moment a concrete name is unavoidable, quarantined to one file.

6.2 Use cases where a factory earns its keep

- **Runtime configuration decides the class** — `cache.policy=lru` in a properties file; the JDBC URL choosing the driver. (You can't `new` from a string without *someone* doing this mapping.)
- **Construction = multi-object assembly** — our cache; a `Connection` needing socket + auth + protocol handshake.
- **The caller must not know implementations exist** — JDBC again: your code compiles against `java.sql` with zero vendor imports.
- **Instance control** — a factory may return a *cached* instance, a pooled one, or a singleton; a constructor must always return a fresh object. (`Integer.valueOf(42)` returns a cached Integer; `Boolean.valueOf` never allocates at all.)
- **Readable, intent-named creation** — `Executors.newFixedThreadPool(8)`, `List.of(1,2,3)`, `Duration.ofMinutes(5)`: names say *what you're making*, constructors can't.

You already use factories fluently: `List.of`, `Optional.empty`, `Stream.of`, `Executors.new*`, `Files.newBufferedReader`, `Charset.forName`, `DriverManager.getConnection`, and — from Notes 02 — every `from(ctx)` on our DTOs.

6.3 Questions students always ask (asked for you, answered)

Q1. "The switch inside the factory — isn't that the exact conditional we just removed?" Yes. Reread §5.3: the win was never zero conditionals; it was *one site, run once, at the edge*. And the factory's switch can become the `Map<String, Supplier<...>>` registry — data, not code. The domain stays conditional-free either way.

Q2. "When is a plain constructor fine?" When the caller may legitimately know the concrete class: inside the same package, in the composition root, for value objects (`new Position(15, 42)`). Factory ceremony for a record you own and see is over-engineering. The trigger for a factory is *the caller shouldn't choose or shouldn't know* — or creation has a recipe.

Q3. "Static factory method vs Factory Method pattern vs Abstract Factory — same thing?" Three related things, one family:

Term	What it is	Example
Static factory method	a static method that creates; the everyday workhorse	<code>List.of</code> , <code>Caches.of</code> , <code>QuestionFilter.from(ctx)</code>
Factory Method (GoF)	creation deferred to a subclass override — a template-method for <code>new</code>	<code>Iterable.iterator()</code> : each collection creates <i>its</i> iterator
Abstract Factory (GoF)	an object that creates whole <i>families</i> of related things	a <code>StorageFactory</code> making matching <code>CacheStorage</code> + <code>Lock</code> + <code>Serializer</code> for "on-heap" vs "off-heap"

In day-to-day code, "factory" almost always means the first. Know the other two mostly to read old codebases and pass interviews.

Q4. "Is a DI container just a big factory?" Essentially yes — Spring is a factory that reads the mapping (annotations/config) and does the assembly (injection) for the whole application graph. Our hand-written composition root *is* the factory pattern applied to `main`. Once you've written `Caches.of`, `@Bean` will never look like magic.

Q5. "Where does the factory live, and may it be reused across the app?" At the edge, with the other concrete-aware code (composition root's neighborhood). It's the *quarantine zone* for `new` — the rest of the codebase imports interfaces only. Reuse freely; it holds no state.

Q6. "Should the factory return `Cache` or `InMemoryCache`?" The interface — always. Returning the concrete type re-leaks the knowledge the factory exists to contain. (Corollary: if there's only one implementation and never will be another, ask whether you needed the factory at all — see Q2.)

7. Adding TTL — Decorator, or Something Better?

The new requirement: entries expire after a time-to-live. Two candidate designs — and choosing between them teaches you exactly what decorators are for.

7.1 Candidate A — a TTL decorator

We know this move from the repository wrappers: implement `Cache`, wrap a `Cache`, add behavior around the calls:

```

public class TtlCache<K, V> implements Cache<K, V> {

    private final Cache<K, V> inner;           // wraps ANY cache
    private final Map<K, Instant> deadlines = new HashMap<>();
    private final Duration ttl;
    private final Clock clock;                // injected – tests control time

    public TtlCache(Cache<K, V> inner, Duration ttl, Clock clock) {
        this.inner = inner; this.ttl = ttl; this.clock = clock;
    }

    @Override
    public V get(K key) {
        Instant deadline = deadlines.get(key);
        if (deadline != null && clock.instant().isAfter(deadline)) {
            inner.remove(key);           // lazy expiration: expire on read
            deadlines.remove(key);
            return null;
        }
        return inner.get(key);
    }

    @Override
    public void put(K key, V value) {
        deadlines.put(key, clock.instant().plus(ttl));
        inner.put(key, value);
    }
    // remove: also clear deadline; others delegate ...
}

```

Attractive: `InMemoryCache` untouched, stacks with other wrappers, works over any `Cache`. But probe it and cracks appear — all of the same kind:

1. **size() lies.** Expired-but-not-yet-read entries still count. The decorator can't fix it without scanning every deadline on every `size()` call.
2. **State drifts apart.** When the *inner* cache evicts a key at capacity, the decorator's `deadlines` map never hears about it — it leaks entries for keys that no longer exist. Two components each own half the truth about "what's in the cache."
3. **The eviction policy can't see expiration.** A full cache evicts the LRU victim — while three *already-expired* entries sit there. The obviously-right victim is invisible to the policy, because expiration lives outside the wall.
4. **Per-entry TTL needs a new method** (`put(k, v, ttl)`), and a decorator cannot add methods to an interface it merely implements — you'd need `CacheWithTTL` extends `Cache`, and now every wrapper in the stack must implement the fat interface too.

Diagnosis: **TTL is not orthogonal to the cache — it interacts with size, capacity, and eviction.** And that's the boundary of the decorator pattern in one sentence:

Decorate what's orthogonal (logging, metrics, stats — behaviors that observe calls without changing the core's meaning). **Compose what interacts** (behaviors that other internals must be able to see).

7.2 Candidate B — ExpirationPolicy as a third composed strategy (the better design)

Expiration is a *policy*, exactly like eviction. Give it the same shape and inject it beside the others:

```

public interface ExpirationPolicy<K> {
    void onPut(K key);           // start the clock for this key
    boolean isExpired(K key);
    void onRemove(K key);       // stop tracking
}

public class TtlExpirationPolicy<K> implements ExpirationPolicy<K> {
    private final Map<K, Instant> deadlines = new HashMap<>();
    private final Duration ttl;
    private final Clock clock;

    public TtlExpirationPolicy(Duration ttl, Clock clock) { this.ttl = ttl; this.clock = clock; }

    public void onPut(K key) { deadlines.put(key, clock.instant().plus(ttl)); }
    public boolean isExpired(K key) {
        Instant d = deadlines.get(key);
        return d != null && clock.instant().isAfter(d);
    }
    public void onRemove(K key) { deadlines.remove(key); }
}

public class NeverExpires<K> implements ExpirationPolicy<K> { // Null Object – again!
    public void onPut(K key) { }
    public boolean isExpired(K key) { return false; }
    public void onRemove(K key) { }
}

```

The cache consults it *inside* the wall, where all the state lives together:

```

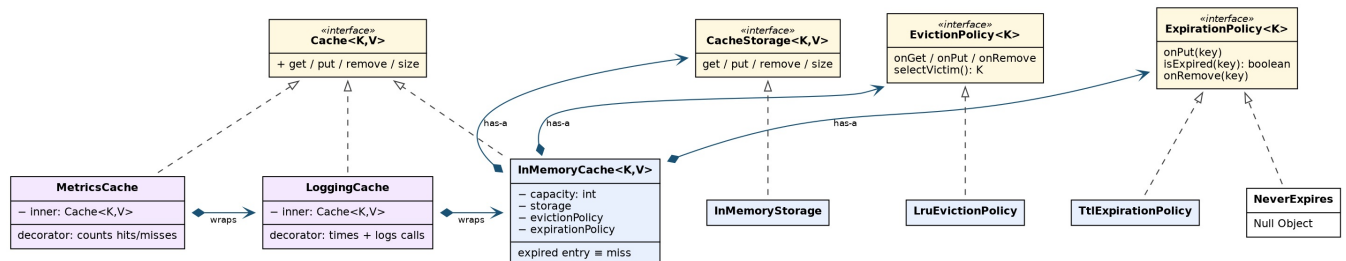
public class InMemoryCache<K, V> implements Cache<K, V> {

    private final CacheStorage<K, V> storage;
    private final EvictionPolicy<K> evictionPolicy;
    private final ExpirationPolicy<K> expirationPolicy; // third strategy, same idiom
    // constructor injects all three ...

    @Override
    public V get(K key) {
        if (expirationPolicy.isExpired(key)) {
            evictAsExpired(key); // storage AND policies all told
            return null;        // an expired entry IS a miss
        }
        V value = storage.get(key);
        if (value != null) evictionPolicy.onGet(key);
        return value;
    }

    private void evictAsExpired(K key) {
        storage.remove(key);
        evictionPolicy.onRemove(key); // ← the fix for cracks 2 & 3:
        expirationPolicy.onRemove(key); // everyone hears about it
    }
}

```



Final architecture — three composed policies inside the wall; logging and metrics as decorators outside it.

Every crack sealed: size can be made honest, no split-brain state, eviction can be taught to prefer expired victims, per-entry TTL is a policy-level feature (`onPut(key, ttl)`) that doesn't disturb the wrappers. This is precisely how **Caffeine** treats expiration — `expireAfterWrite` is a configured first-class policy inside the cache, while its stats recording (`recordStats`) is wrapper-style instrumentation around it. The library agrees with the rule.

(Design footnote: pure lazy expiration means an untouched key lingers until read. Real caches add a background sweep or piggyback cleanup on writes — mention it in an interview, then say "out of scope for v1" like a professional.)

So the final answer to the class question: yes, a decorator can bolt TTL on, and for a quick, approximate cache it even works. But TTL fails the orthogonality test — so the better design is what we already had: **one more strategy, composed in**. Logging and metrics remain decorator territory:

```
Cache<Long, Question> cache =
    new MetricsCache<>() // decorator: orthogonal, observes
    new LoggingCache<>() // decorator: orthogonal, observes
    new InMemoryCache<>(10_000,
        new InMemoryCacheStorage<>(),
        new LruEvictionPolicy<>(),
        new TtlExpirationPolicy<>(Duration.ofMinutes(5), Clock.systemUTC())));
```

One expression, five patterns, zero `extends` .

8. Summary

- Cache design 1** (class-per-policy + factory switch) works but duplicates storage, mixes concerns, and explodes $N \times M$ when features arrive. **Design 2** composes: `Cache` HAS-A `CacheStorage` + `EvictionPolicy` (hooks: `onGet` / `onPut` / `onRemove` , `selectVictim`) — Strategy for the volatile part, DI at the constructor, $N + M$ classes.
- LRU mechanics:** `HashMap` → node references ⇌ doubly linked list; $O(1)$ get, reorder, and evict. `LinkedHashMap(accessOrder=true)` is the JDK's built-in version.
- Composition over inheritance:** inheritance couples you to a parent's *internals* (fragile base class — `InstrumentedHashSet` counts 6, not 3) and is fixed at compile time; composition couples to *interfaces*, swaps at runtime, combines freely, and tests in isolation. The JDK's own regrets (`Stack` extends `Vector` , `Properties` extends `Hashtable`) are the cautionary tales; `Collections.unmodifiableList` and Caffeine are the success stories. **Inherit for substitution, compose for reuse.**
- Replace conditionals with polymorphism — honestly:** the conditional doesn't vanish; it moves from *N sites, every call, as control flow* to *one site, once, as data* — and even that site can become a `Map` registry. Polymorphic dispatch is itself a compiler-maintained jump table. Keep a small exhaustive `switch` when cases are few and the site is single; go polymorphic when cases or sites multiply.
- Factory:** quarantine for `new` . It owns the intent→implementation mapping and the assembly recipe, returns the interface, and is DIP applied to the moment of creation. JDBC's `DriverManager` is the everyday proof; a DI container is this pattern grown up.
- TTL:** the decorator version leaks (size lies, split-brain state, eviction blind to expiry, can't add methods) because TTL **interacts** with the cache's internals. The rule: **decorate the orthogonal, compose the interacting** — so `ExpirationPolicy` joins storage and eviction as a third injected strategy, exactly as Caffeine does.

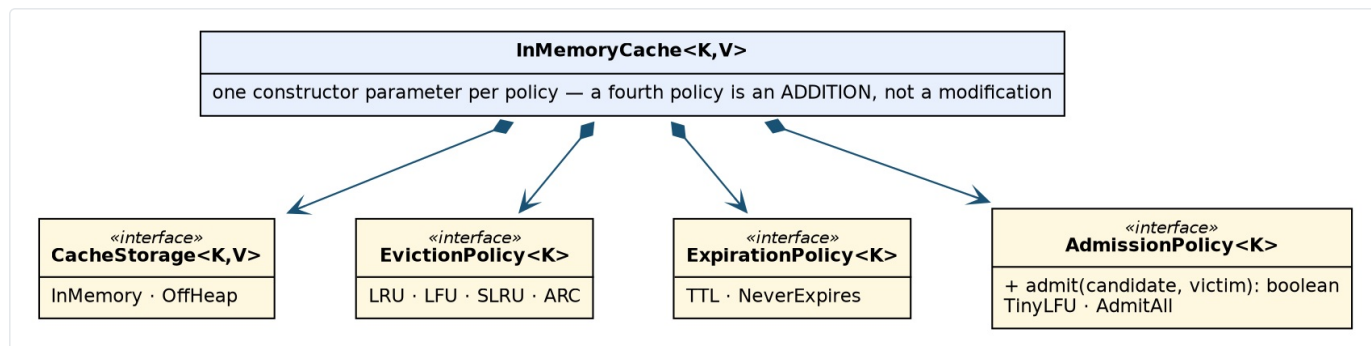
Appendix — Beyond LRU: the Eviction Policy Zoo

Proof that the `EvictionPolicy` abstraction earns its keep: all of these plug into our cache without touching a line of it. Five worth knowing, in a natural difficulty progression (**LRU** → **LFU** → **SLRU** → **TinyLFU** → **ARC**):

Policy	Idea	Why it's interesting
Second Chance / Clock	Each entry gets a reference bit; on eviction scan: bit=1 → clear it, spare it; bit=0 → evict	LRU's benefits without maintaining exact order — $O(1)$ with a circular pointer; how OS page replacement works
SLRU (Segmented LRU)	Two LRU segments: new entries enter <i>probationary</i> ; a second access promotes to <i>protected</i>	One big scan of a million one-time keys can no longer flush your entire useful cache
TinyLFU	Keeps approximate frequency counts; a newcomer must <i>beat the victim's frequency</i> to get in	Introduces a new idea entirely — an admission policy: "should this even enter?" is a different question from "who leaves?"
W-TinyLFU	TinyLFU admission + a small LRU window for recent items	What Caffeine actually runs — near-optimal hit rates in practice
ARC	Adaptively splits capacity between a recency list and a frequency list based on the live workload	Self-tuning: stops the "LRU or LFU?" argument by measuring instead of guessing

The TinyLFU row hides one more architectural lesson — admission and eviction are **separate decisions**, so they get separate abstractions:

```
public interface AdmissionPolicy<K> {
    boolean admit(K candidate, K victim); // full cache: does the newcomer deserve the spot?
}
```



A fourth policy (admission) arrives — and the design absorbs it as one more constructor parameter.

Notice what just happened: a fourth policy arrived and the design absorbed it the same way as the third — one interface, one constructor parameter, a Null-Object default (`AdmitAll`). When new requirements keep landing as *additions* rather than *modifications*, that's the design telling you the decomposition was right.